

TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics

Software Engineering

OPERATING SYSTEMS

Laboratory Work No. 13

Semaphores in Your Own Kernel (xv6)

Performed by:
Bianca-Andreea Tataroi
Group FAF-242

Checked by:
Patricia Reițman

Chisinau, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Part 1 – The Numbers and the User Side **2**
3.1 Preparing and Booting xv6 2
3.2 Adding System Calls and Test Programs 3
3.3 Check 1 – System Calls Not Yet Implemented 3

4 Part 2 – The Kernel Side **4**
4.1 Implementation of the Semaphore Table 4
4.2 Kernel Registration and Successful Test 4

5 Part 3 – A Semaphore as a Lock: mix **5**
5.1 Execution Without a Lock 5
5.2 Execution With a Lock 6

6 Answers to Observation Questions **6**

7 Conclusion **7**

8 GitHub Repository **7**

1 Introduction

Process synchronization is essential when several processes share resources. Without coordination, concurrent processes may execute in an unexpected order or interleave operations on a shared device. In previous laboratory works, semaphore functions provided by a C library were used. In this laboratory, three semaphore system calls were implemented directly in the educational xv6 kernel: `sem_init`, `sem_down`, and `sem_up`. Two user programs, `semtest` and `mix`, were used to demonstrate process ordering, ping-pong synchronization, and mutual exclusion.

2 Objectives

The objectives of this laboratory work were:

- Add semaphore system call numbers 24, 25, and 26 and expose them to xv6 user programs.
- Implement eight kernel-level counting semaphores protected by spinlocks.
- Use kernel sleep and wakeup mechanisms to block and resume processes.
- Verify parent-child execution ordering and alternating ping-pong output.
- Compare concurrent terminal output without synchronization and with a semaphore lock.

3 Part 1 – The Numbers and the User Side

3.1 Preparing and Booting xv6

The xv6 source tree was prepared on Ubuntu, and the system was started using `make qemu`. The kernel booted and started its shell successfully.

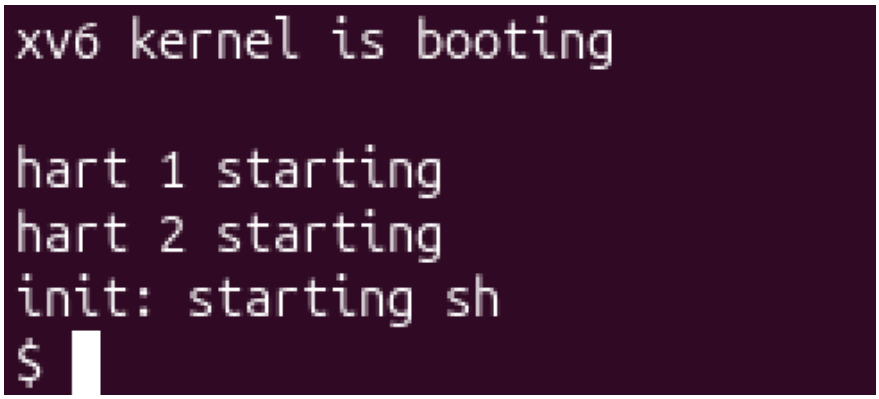


Figure 1: The xv6 kernel boots and opens the shell successfully

3.2 Adding System Calls and Test Programs

The three new system call numbers were added to `kernel/syscall.h`. Corresponding declarations were added to `user/user.h`, and user-space stubs were registered in `user/usys.pl`.

```
#define SYS_sem_init 24
#define SYS_sem_down 25
#define SYS_sem_up 26
```

The program `user/semtest.c` tests the execution order of a parent and child process and then uses two semaphores to alternate ping and pong. The program `user/mix.c` prints characters from two processes, with an optional semaphore lock. Both programs were added to the `UPROGS` list in the Makefile.

3.3 Check 1 – System Calls Not Yet Implemented

Before implementing the kernel handlers, the `semtest` program was executed. The kernel displayed `unknown sys call` 24, 25, and 26, proving that the user code invoked the new syscall numbers but that no kernel implementation was registered yet.

```
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
3 semtest: unknown sys call 26
3 semtest: unknown sys call 24
3 semtest: unknown sys call 24
3 semtest: unknown sys call 25
pi5 semtest: unknown sys call 25
ng 1
3  pong 1
 semtest: unknown sys call 26
3 semtest: unknown sys call 25
ping 2
3 semtest: unknown sys call 26
3 semtest: unknown sys call 25
ping 5 semtest: unknown sys call 26
5 semtest: unknown sys call 25
3
3  pong 2
 semtest: unknown sys call 26
5 semtest: unknown sys call 26
5 semtest: unknown sys call 25
  pong 3
5 semtest: unknown sys call 26
semtest: OK
```

Figure 2: Initial semaphore test before the kernel handlers were implemented

The child message appeared before the parent message, and some ping-pong text was interleaved. Although the program printed `semtest: OK`, synchronization was not yet functional: this final message only indicated that the program reached its last statement.

4 Part 2 – The Kernel Side

4.1 Implementation of the Semaphore Table

The new file `kernel/sem.c` defines eight counting semaphores shared between processes. Each semaphore holds an integer `value` and a spinlock that protects that integer against concurrent updates.

```
#define NSEM 8
struct sem {
    struct spinlock lock;
    int value;
};
static struct sem sems[NSEM];
```

The `sys_sem_init` function initializes a selected semaphore to a nonnegative value. The `sys_sem_down` function decreases that value when a unit is available; otherwise, it registers the wait channel, releases the spinlock, and sleeps. The `sys_sem_up` function increments the value and calls `wakeup` to resume waiting processes.

```
acquire(&s->lock);
while (s->value == 0) {
    sleep_prepare(s);
    release(&s->lock);
    sleep();
    acquire(&s->lock);
    if (killed(myproc())) {
        release(&s->lock);
        return -1;
    }
}
s->value--;
release(&s->lock);
```

The implementation uses the course's two-step `sleep_prepare()` and `sleep()` interface. The `while` loop is necessary because a process can awaken after another process has already consumed the available semaphore unit.

4.2 Kernel Registration and Successful Test

The object `$K/sem.o` was included in the Makefile, and the three handlers were declared and registered in `kernel/syscall.c`. After rebuilding and booting xv6, `semtest` produced the intended execution order.

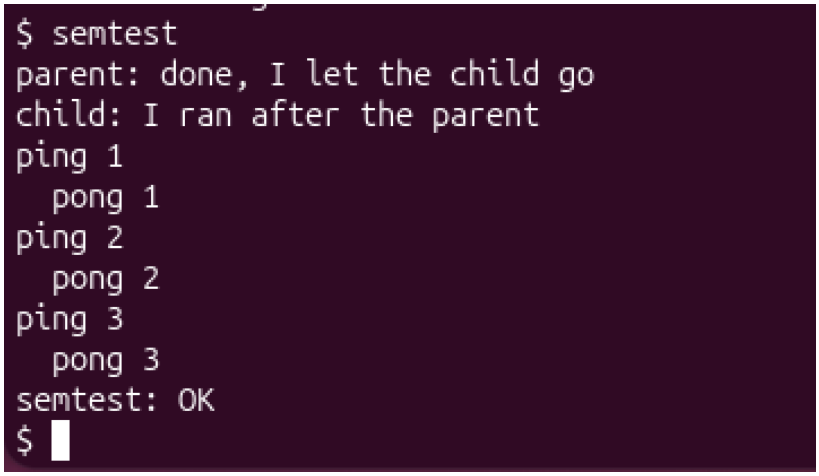


Figure 3: Successful parent-child synchronization and alternating ping-pong execution

The parent printed before the child even though the child was started first and waited on semaphore 0. The second test printed `ping 1`, `pong 1`, through `ping 3`, `pong 3`, in strict alternation. The test concluded with `semtest: OK`, demonstrating the required behavior.

5 Part 3 – A Semaphore as a Lock: mix

5.1 Execution Without a Lock

The command `mix` was executed repeatedly. The parent wrote lines containing `A`, while the child wrote lines containing `b`. Each character was written with a separate `write()` system call, allowing the processes' outputs to interleave.

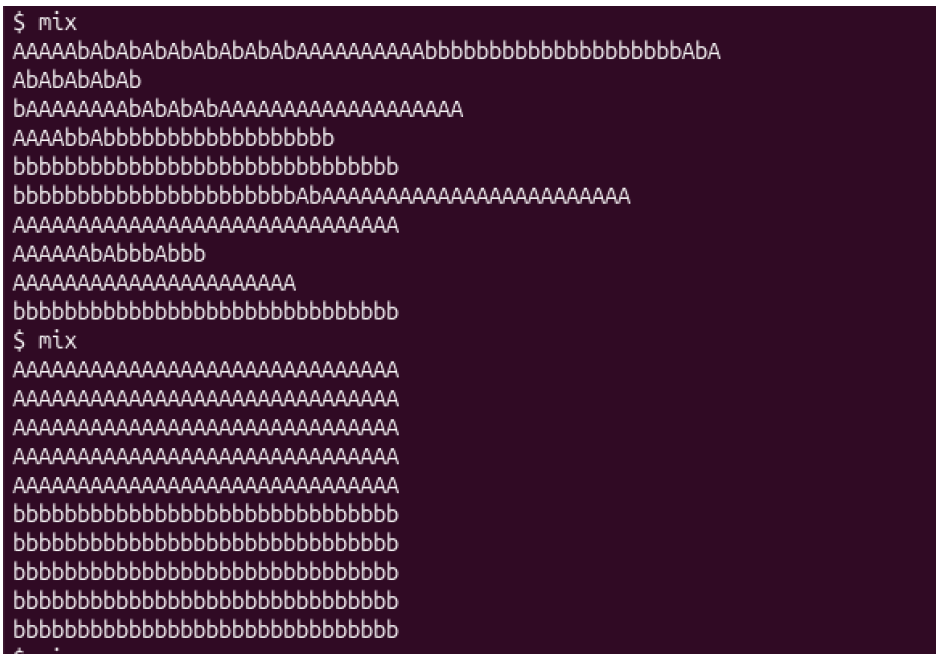


Figure 4: Two executions of mix without a lock: output depends on scheduling

In the first execution, lines contained mixed `A` and `b` characters. In the second, the lines happened to remain whole. Both results are valid: without synchronization, the scheduler may or may not interrupt a process while it prints a line.

5.2 Execution With a Lock

The command `mix lock` used semaphore 3, initialized to 1, to protect the complete printing of each line.

```
if (uselock)
    sem_down(3);
for (char *p = s; *p; p++)
    write(1, p, 1);
if (uselock)
    sem_up(3);
```

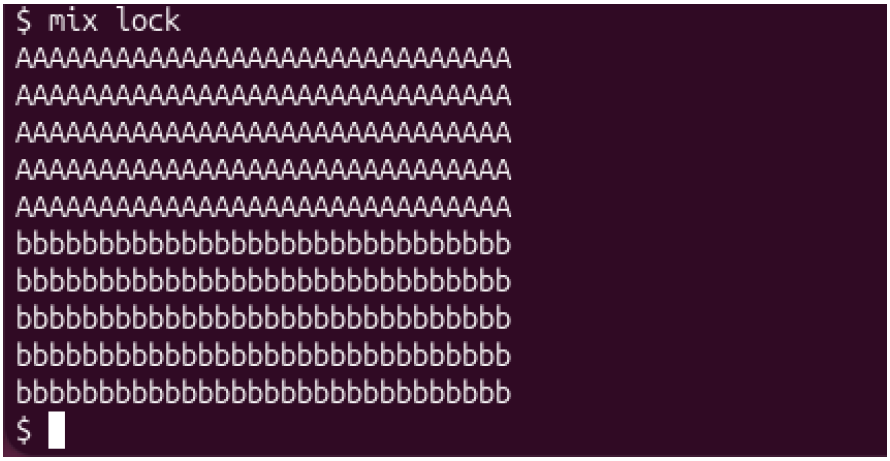


Figure 5: mix lock prints five complete A lines and five complete b lines

The resulting output contained five complete lines of each character, with no mixed lines. This demonstrates mutual exclusion: each process holds the semaphore while printing its entire line.

6 Answers to Observation Questions

1. **Without the lock, why do the letters mix? How many system calls does one line cost?**
   The two processes run concurrently and each character is printed using its own `write()` call. Each line contains 30 characters and a newline, requiring 31 system calls. The scheduler can switch between processes between calls, allowing their output to mix.

2. **Which two lines of mix.c make one line a critical section?**
   The lines `sem_down(3);` and `sem_up(3);` surround the character-printing loop. The first obtains the semaphore and the second releases it after the whole line is written.

3. **Why does sem_down use while instead of if?**
   Waking up does not guarantee that a semaphore unit remains available. Another

process might acquire it first. Rechecking with `while` prevents a process from passing when the semaphore value is still zero.

4. **Why must the spinlock be released before sleep()?**
   A sleeping process must not retain a spinlock. Otherwise, another process may be unable to acquire that lock to update the semaphore and signal the sleeper, leading to deadlock.

7 Conclusion

In this laboratory work, counting semaphores were implemented directly in the xv6 kernel by adding three system calls and a shared semaphore table. The initial test showed that unimplemented calls could not coordinate processes. After implementing kernel sleep, wakeup, and spinlock protection, the `semtest` program demonstrated correct parent-child ordering and strict ping-pong alternation. The `mix` experiment further demonstrated that concurrent character writes can interleave without a lock, while semaphore-based mutual exclusion preserves complete lines. The screenshots confirm the expected behavior of all required experiments.

8 GitHub Repository

The xv6 source tree for this laboratory includes `kernel/sem.c`, the modified syscall registration files, `user/semtest.c`, `user/mix.c`, the Makefile, and `notes.txt`. The intended repository URL is:

https://github.com/Biancatataroi/Operating-Systems-Lab13